Стыковка движка захвата с другими стадиями

Английский оригинал: INTEGRATION.md.

Движок захвата — первая стадия конвейера. Его единственный выход — нормализованный JSON захвата, следующий docs/CONTRACT.md. Стадия протокола и стадия интерфейса обе потребляют этот один файл. Ни одна стадия не читает PCAP напрямую.

```
PCAP/PCAPNG  --движок захвата-->  нормализованный JSON  --+--> стадия протокола (правила, гипотезы)
                                                          +--> стадия интерфейса (сессии, hex, провенанс)
```

Передача захвата стадии протокола

Стадия протокола работает по нормализованному JSON, а не по захвату. Получите его через CLI:

```
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
    --out out/corpus_01.json --source-name captures/corpus_capture_01.pcapng
```

Байты потока для каждого направления лежат в sessions[].directions.<A_to_B|B_to_A>.bytes_b64, и каждый диапазон байт имеет запись провенанса. Это всё, что нужно, чтобы сфреймить сообщения и вернуть результат поля к пакету.

Передача захвата стадии интерфейса

Стадия интерфейса читает тот же файл. Список сессий, конечные точки, диагностики и диапазоны провенанса напрямую отображаются на дерево сессий, hex-просмотр и панель деталей пакета. Интерфейс не должен разбирать захват заново.

Сценарий проверки

Это сквозная проверка шва между движком захвата и двумя другими стадиями. Запускайте её после любого изменения модуля захвата.

1. Сгенерируйте корпус (детерминированно; повторять безопасно):

```
python -m scripts.generate_corpus
```

1. Нормализуйте основной захват корпуса:

```
mkdir -p out
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
    --out out/corpus_01.json --source-name captures/corpus_capture_01.pcapng
```

1. Проверьте выход по контрактной схеме:

```
python - <<'PY'
import json
from jsonschema import Draft202012Validator

schema = json.load(open("docs/schemas/capture.schema.json"))
data = json.load(open("out/corpus_01.json"))
Draft202012Validator(schema).validate(data)
print("ok:", len(data["sessions"]), "sessions")
PY
```

У основного захвата три сессии, каждая с заполненными обоими направлениями. Потоки должны нести байты записанных транзакций, а каждый диапазон провенанса — разрешаться в настоящий индекс пакета.

1. Свяжите транзакцию с проводом. Возьмите строку из `tests/corpus/corpus_journal.md`, например:

```
1700000000.004 | read | temperature | 1595
```

Найдите пакет с этой временной меткой в захвате и через `provenance.lookup` перейдите от смещения потока назад к этому пакету (см. `docs/CAPTURE_API.md`, раздел «Чтение провенанса»).

1. Откройте JSON в интерфейсе, когда стадия интерфейса готова. Дерево сессий должно перечислить три сессии, hex-просмотр — отрисовать байты потока с провенансом, а захват дефектов — показать свой пропуск и неоднозначное перекрытие как диагностики.

Захват с дефектами

`tests/corpus/corpus_capture_defects.pcapng` несёт случаи, которые нижележащие стадии обязаны обрабатывать честно:

- повторно переданный запрос (лишний провенанс, без лишних байтов),
- запрос не по порядку (переупорядочен по последовательности),
- запрос с недостающим сегментом (диагностика `gap`, направление ответа),
- пара байт ответа, которой позже противоречат (`ambiguity`).

Когда стадия протокола фреймит этот захват, затронутые сообщения должны классифицироваться как `incomplete` или `ambiguous`, никогда `matched`.

Границы

- Движок захвата не интерпретирует байты. Он сообщает только наблюдения и провенанс.
- Стадии протокола и интерфейса не читают PCAP. Если любой из них нужны байты, которых JSON не несёт, это изменение контракта, и его надо согласовать с участником.

Эталонная выгрузка для стадии протокола

`tests/corpus/reference_export/` держит нормализованную выгрузку трёх сгенерированных записей корпуса, закоммиченную, чтобы стадия протокола и интерфейс могли построить свой дымовой тест корпуса и эталонное исследование, не вызывая модуль захвата вообще.

Файл	Сессий	Несёт
`corpus_capture_01.normalized.json`	3	чистый трафик чтения/записи/измерения
`corpus_capture_02.normalized.json`	2	тот же протокол, другие значения
`corpus_capture_defects.normalized.json`	1	`gap` в `A_to_B`, `ambiguity` в `B_to_A`

Пересобрать их из захватов:

```
bash tests/corpus/reference_export/regenerate.sh
```

Файлы байт-в-байт воспроизводимы из данного коммита. `tests/integration/test_reference_export.py` падает, если закоммиченный файл перестаёт совпадать с тем, что производит модуль захвата, поэтому стадия протокола никогда не работает против устаревшей выгрузки. Правило, написанное против потока из `corpus_capture_01`, должно проверяться на `corpus_capture_02` (тот же протокол, другие значения) на контрпримеры и на `synthetic_live` (другая раскладка байт), чтобы подтвердить, что оно не верно случайно.

Формат проекта на диске

Исследование — каталог, создаваемый `src.project`:

```
project.madrigal/
  manifest.json          индекс всего ниже, все пути относительные
  captures/              исходные захваты, скопированные внутрь
  logs/                  журнал действий
  rules/                 правила интерпретации, один файл на версию
  results/               результаты применения правил
  annotations.sqlite     заметки исследователя
  reports/               сгенерированные отчёты Markdown и HTML
```

`manifest.json` записывает `contract_version`, `project_version`, `name`, `created_at`, `updated_at`, `captures` (относительный путь + `sha256`), `rules` (id + версия + путь + `sha256`), `results` и `annotations`. Поскольку каждый путь относительный, а каждый захват несёт свою контрольную сумму, каталог можно скопировать в другой каталог или на другую машину и открыть там без изменений:

```
from pathlib import Path
from src.project import Project

project = Project.create(Path("work/project.madrigal"), "cap1 investigation")
project.add_capture(Path("captures/cap1.pcapng"))
project.add_rule(Path("rules/set_parameter.json"), "set_parameter_request", 3)
project.export(Path("work/cap1.zip"))

# На другой машине:
restored = Project.import_(Path("work/cap1.zip"), Path("work/restored"))
assert restored.verify_captures() == []  # контрольные суммы захватов совпадают
assert restored.verify_rules() == []  # контрольные суммы правил совпадают
```

`Project.import_` проверяет каждый захват и каждое зарегистрированное правило по записанной контрольной сумме и отвергает подделанный архив, удаляя частичный каталог. См. раздел проекта в `docs/CONTRACT.md`.

Шов отчёта

Стадия протокола строит словарь исследования, а модуль отчёта его отрисовывает. Ключи словаря — `hypotheses`, `counterexamples`, `rule_versions`, `scope` и `open_questions` (точную форму см. в `src/report/model.py`). Отрисуйте обе формы:

```
from pathlib import Path
from src.report import render_html, render_markdown

render_markdown(investigation, Path("reports/investigation.md"))
render_html(investigation, Path("reports/investigation.html"))
```

Отчёт Markdown — результат сдачи; HTML-страница — то же содержимое, оформленное по `docs/STYLE.md` и самодостаточное, поэтому отрисовывается без доступа к сети.